

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №5
по дисциплине «Построение и анализ алгоритмов»
ТЕМА: Ахо-Корасик

Студент гр. 4343

Миридаштаки Ф.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург

2026

Цель работы.

Изучить принцип работы алгоритма Ахо-Корасик. Написать две программы, решающие задачи поиска набора образцов в тексте и одного образца с джокером.

Задание №1.

Разработайте программу, решающую задачу точного поиска набора образцов.

Входные данные:

Первая строка содержит текст ($T, 1 \leq |T| \leq 100000$).

Вторая - число n ($1 \leq n \leq 3000$), каждая следующая из n строк содержит шаблон из набора $P = \{p_1, \dots, p_n\}$ $1 \leq |p_i| \leq 75$

Все строки содержат символы из алфавита $\{A, C, G, T, N\}$

Выходные данные:

Все вхождения образцов из P в T .

Каждое вхождение образца в текст представить в виде двух чисел - i p

Где i - позиция в тексте (нумерация начинается с 1), с которой начинается вхождение образца с номером p (нумерация образцов начинается с 1).

Строки выхода должны быть отсортированы по возрастанию, сначала номера позиции, затем номера шаблона.

Sample Input:

NTAG

3

TAGT

TAG

T

Sample Output:

2 2

2 3

Задание №2.

Используя реализацию точного множественного поиска, решите задачу точного поиска для одного образца с *джокером*.

В шаблоне встречается специальный символ, именуемый джокером (wild card), который "совпадает" с любым символом. По заданному содержащему шаблоны образцу P необходимо найти все вхождения P в текст T .

Например, образец $ab??c?$ с джокером $?$ встречается дважды в тексте $xabvccbababcax$.

Символ джокер не входит в алфавит, символы которого используются в T . Каждый джокер соответствует одному символу, а не подстроке неопределённой длины. В шаблон входит хотя бы один символ не джокер, т.е. шаблоны вида $???$ недопустимы.

Все строки содержат символы из алфавита $\{A,C,G,T,N\}$

Входные данные:

Текст ($T, 1 \leq |T| \leq 100000$)

Шаблон ($P, 1 \leq |P| \leq 40$)

Символ джокера

Выходные данные:

Строки с номерами позиций вхождений шаблона (каждая строка содержит только один номер).

Номера должны выводиться в порядке возрастания.

Sample Input:

ACTANCA

A\$\$\$A\$

\$

Sample Output:

1

Вариант 5: Вычислить максимальное количество дуг, исходящих из одной вершины в боре; вырезать из строки поиска все найденные образцы и вывести остаток строки поиска.

Описание алгоритма Ахо-Корасик

Алгоритм основан на построении префиксного дерева (бора) из заданных паттернов с последующим обходом переданного текста. На входе известны только сами паттерны.

На первом этапе строится бор из символов паттернов. После этого в него добавляются суффиксные и терминальные ссылки. Суффиксная ссылка каждой вершины указывает на вершину, соответствующую наидлиннейшему собственному суффиксу текущей строки, который присутствует в боре. Корень бора в качестве исключения ссылается на себя. Суффиксные ссылки необходимы для случаев, когда при обработке очередного символа текста переход по боровым рёбрам невозможен — либо паттерн найден полностью, либо нужного ребра просто нет. Помимо суффиксных, существуют терминальные ссылки, которые указывают на ближайшую терминальную вершину, достижимую по цепочке суффиксных ссылок.

После построения автомата выполняется поиск в тексте. Каждый символ текста вызывает переход из текущего состояния в новое — либо по ребру бора, либо по суффиксной ссылке. Благодаря этому автомат никогда не «застревает»: даже достигнув конца паттерна, следующий переход по суффиксной ссылке вернёт в корень или в более раннюю вершину. Терминальные ссылки позволяют эффективно обнаруживать все вхождения, не пропуская паттерны, которые

являются суффиксами других паттернов. В итоге алгоритм находит все вхождения всех паттернов в тексте.

Сложность по времени:

Временная сложность алгоритма составляет $O(\sum |p_i| + |T|)$, где $\sum |p_i|$ — суммарная длина всех паттернов, а $|T|$ — длина текста. Построение бора занимает $O(\sum |p_i|)$, поскольку каждый символ каждого паттерна обрабатывается ровно один раз. Обход текста для поиска вхождений выполняется за $O(|T|)$, так как из каждого состояния автомата переход происходит за константное время благодаря заранее построенным суффиксным и терминальным ссылкам.

Сложность по памяти:

Сложность алгоритма по памяти - $O(\sum |p_i| * k)$, где k — размер алфавита, поскольку каждая вершина хранит множество своих детей, и в худшем случае его дети — все символы данного алфавита.

Описание модифицированного алгоритма

Построение бора и суффиксных ссылок выполняется так же, как в основном алгоритме, однако в качестве паттернов используются подстроки шаблона, разделённые символом джокера. Логика поиска этих подстрок в тексте практически не отличается от базового случая. После нахождения всех совпадений получаем список позиций и подстрок, которые потенциально являются частями искомого шаблона. Для каждого такого совпадения вычисляется позиция, с которой должен начинаться весь шаблон, чтобы данная подстрока оказалась на своём месте. По каждой такой позиции ведётся счётчик, который увеличивается на единицу при каждом подходящем совпадении. Если значение счётчика для некоторой позиции равно общему числу подстрок, это означает, что шаблон входит в текст начиная с данной позиции.

Сложность по времени для модифицированного алгоритма:

Сложность алгоритма по времени - $O(\sum |p_i| + |T|)$ совпадает с оригинальным алгоритмом.

Сложность по памяти для модифицированного алгоритма:

Объём используемой памяти составляет $O(\sum |p_i| \cdot k + |T|)$, где k — размер алфавита. По сравнению с оригинальным алгоритмом добавляется дополнительный массив длиной $|T|$, в котором для каждой позиции текста хранится счётчик количества подстрок шаблона, способных начать вхождение с данной позиции.

Описание функций

1. *build_trie* — строит префиксное дерево (бор) на основе переданных паттернов, добавляя узлы для каждого символа и отмечая терминальные вершины.
2. *build_links* — строит суффиксные и терминальные ссылки для всех узлов бора с помощью обхода в ширину (BFS).
3. *print_trie* — выводит визуализацию построенного бора с информацией о каждой вершине.
4. *search* — выполняет поиск всех вхождений паттернов в тексте, используя автомат с суффиксными и терминальными ссылками, выводит пошаговый процесс.
5. *find_wildcard_matches* — реализует поиск шаблона с джокерами в тексте: разбивает шаблон на подстроки, строит автомат, заполняет массив счётчиков и определяет позиции вхождений.
6. *find_matches* — находит в тексте все вхождения безджокерных подстрок шаблона, возвращает список пар (позиция, индекс подстроки).
7. *max_trie_arcs* — вычисляет максимальное количество дуг, исходящих из одной вершины бора (Вариант 5).
8. *cut_patterns* — вырезает из строки поиска все найденные вхождения паттернов и возвращает остаток строки (Вариант 5).

Тестирование.

Таблица 1 – Тестирование алгоритма Ахо-Корасик

Входные данные	Выходные данные
NTAG 3 TAGT TAG T	2 2 2 3
GCATGCAT 3 GCAT CAT AT	1 1 2 2 3 3 5 1 6 2 7 3
AACTGAAC 2 AAC TG	1 1 4 2 6 1

Таблица 2 – Тестирование алгоритма поиска с джокером

Входные данные	Выходные данные
ACTANCA A\$\$\$ \$	1
GCATNGCAT G\$AT \$	1 6
AACTGAACT A\$CT \$	1 6

```

● farid@Farid-M1-Mac src % python3 main.py
Выберите задание (1 или 2):
1
NTAG
3
TAGT
TAG
T
=====
STEP 1: BUILDING TRIE
=====
New node 1: parent=0, edge='T'
New node 2: parent=1, edge='A'
New node 3: parent=2, edge='G'
New node 4: parent=3, edge='T'
Pattern 1 'TAGT' terminates at node 4
Pattern 2 'TAG' terminates at node 3
Pattern 3 'T' terminates at node 1

Trie built: 5 nodes

[VARIANT 5] Max arcs from one trie node: 1
Node(s) with max arcs: [0, 1, 2, 3]

```

Рисунок 1 – Пример вывода построения бора

```

=====
STEP 2: BUILDING AUTOMATON (fail links + terminal links) via BFS
=====
Node 1: fail=0, term=-1, trie_arcs=['A'], patterns=[3]
Node 2: fail=0, term=-1, trie_arcs=['G'], patterns=[]
Node 3: fail=0, term=-1, trie_arcs=['T'], patterns=[2]
Node 4: fail=1, term=1, trie_arcs=[], patterns=[1]

=====
STEP 3: FULL AUTOMATON DESCRIPTION (every node)
=====
Node 0 | trie_children={'T': 1} | go={'T': 1, 'A': 0, 'G': 0} | fail=0 | term=-1 | patterns=[]
Node 1 | trie_children={'A': 2} | go={'A': 2, 'G': 0, 'T': 1} | fail=0 | term=-1 | patterns=[3]
Node 2 | trie_children={'G': 3} | go={'G': 3, 'A': 0, 'T': 1} | fail=0 | term=-1 | patterns=[]
Node 3 | trie_children={'T': 4} | go={'T': 4, 'A': 0, 'G': 0} | fail=0 | term=-1 | patterns=[2]
Node 4 | trie_children={} | go={'A': 2, 'G': 0, 'T': 1} | fail=1 | term=1 | patterns=[1]

```

Рисунок 2 – Пример вывода создания суффиксных и терминальных ссылок

```

=====
STEP 4: SEARCHING TEXT
=====
Text = 'NTAG'

i=1 char='N': state 0 -> 0
i=2 char='T': state 0 -> 1
    [via node 1] Found pattern 3 'T' at position 2
i=3 char='A': state 1 -> 2
i=4 char='G': state 2 -> 3
    [via node 3] Found pattern 2 'TAG' at position 2

```


Рисунок 3 – Пример вывода поиска в тексте

```
● farid@Farid-M1-Mac src % python3 main.py
Выберите задание (1 или 2):
2
ACTANCA
A$$$A$
$

=====
WILDCARD SEARCH
=====
Text:      'ACTANCA'
Pattern:    'A$$$A$'
Wildcard:   '$'

--- Step 1: Splitting pattern into chunks ---
Chunk: 'A' at pattern position 0
Chunk: 'A' at pattern position 3

--- Step 2: Building Aho-Corasick on chunks ---
New node 1: parent=0, edge='A'
Chunk 'A' terminates at node 1
Chunk 'A' terminates at node 1

--- Step 3: Searching text for chunks ---
i=1 char='A': state 0 -> 1
  Chunk 'A' found, pattern would start at 1 (C[0]=1)
i=2 char='C': state 1 -> 0
i=3 char='T': state 0 -> 0
i=4 char='A': state 0 -> 1
  Chunk 'A' found, pattern would start at 1 (C[0]=2)
i=5 char='N': state 1 -> 0
i=6 char='C': state 0 -> 0
i=7 char='A': state 0 -> 1

--- Step 4: Checking counter array ---
Position 1: C[0]=2 == 2 -> MATCH

=====
FINAL RESULTS
=====
1
```

Рисунок 4 – Пример вывода поиска модифицированного алгоритма

```
=====
FINAL RESULTS (sorted by position then pattern number)
=====
2 2
2 3

[VARIANT 5] Remainder after cutting all found patterns: 'N'
```

Рисунок 5 – Пример вывода максимального количества дуг в боре и остатка строки после вырезания найденных образцов

Исследование.

Для проведения исследования, оценим рост временной сложности создания бора и поиска в тексте и сравним с теоретическими данными.

Таблица 3 – Исследование эффективности алгоритма по времени

Суммарный размер паттернов	Время создания бора
10	0.000010
100	0.000041
1000	0.000671
10000	0.009730
100000	0.102186
Размер текста	Время поиска вхождений
100	0.000010
1000	0.000093
10000	0.000976
100000	0.009473
1000000	0.095374

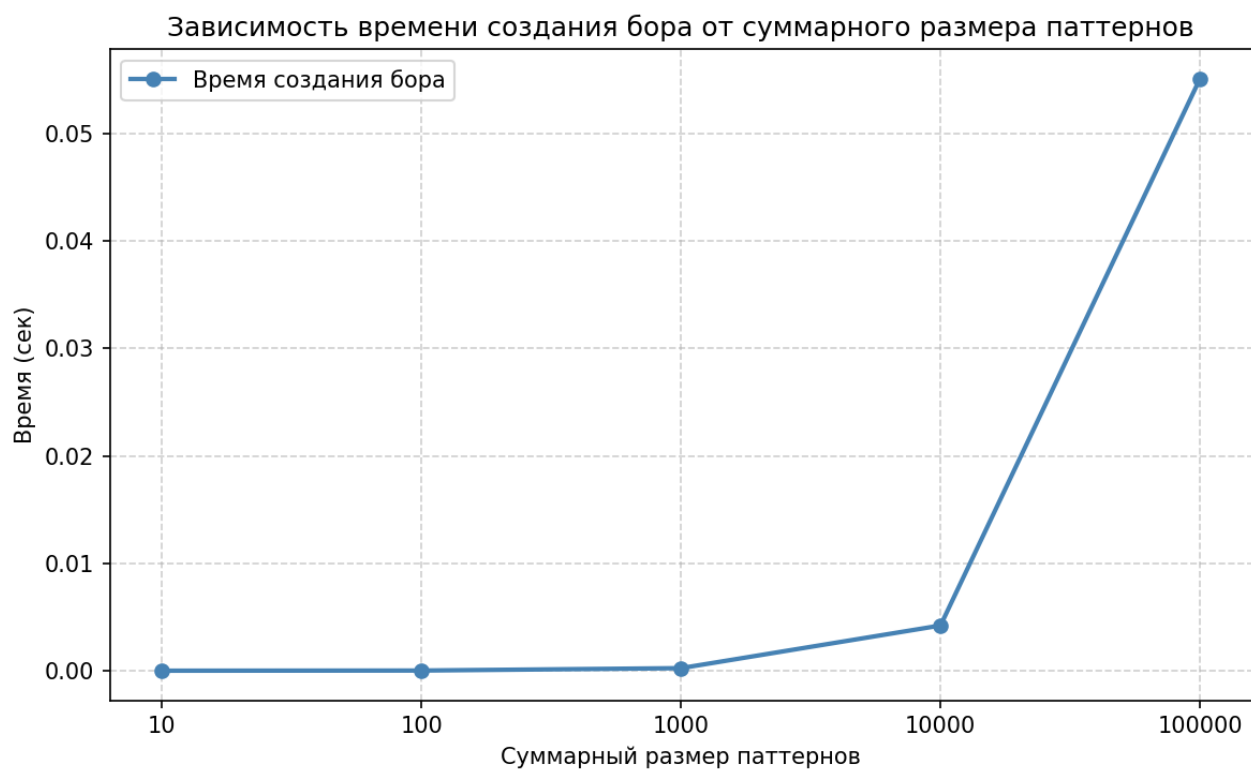


Рисунок 6 – Зависимость времени создания бора от суммарного размера паттернов

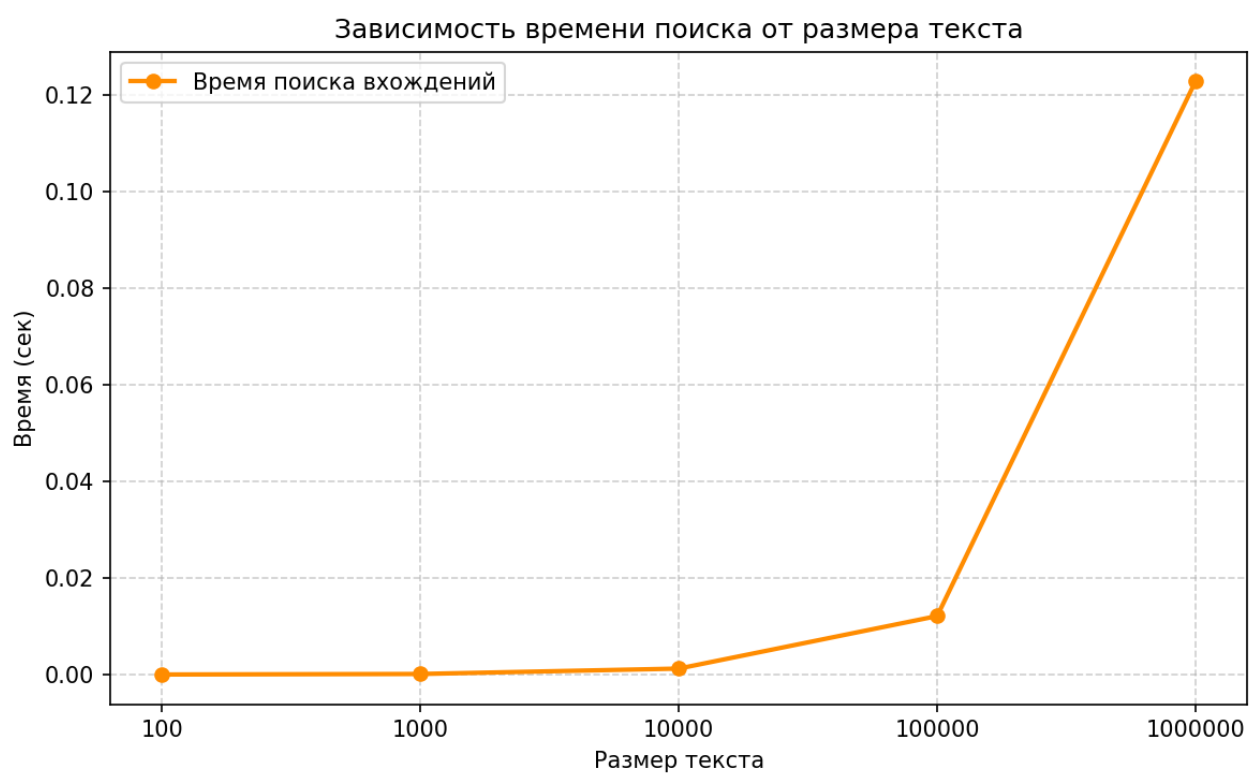


Рисунок 7 – Зависимость времени поиска от размера текста

Можно сделать вывод, что время построения бора и поиска в тексте линейно и зависит соответственно от размеров паттернов и текста, что подтверждает теоретическую оценку. Это можно увидеть в таблице или на графике.

Выводы.

Изучен принцип работы алгоритма Ахо-Корасик. Написаны программы, решающие задачу поиска набора паттернов в тексте и поиска шаблона с джокером в тексте. Дополнительно реализованы задачи вычисления максимального количества дуг, исходящих из одной вершины бора, и вырезания найденных вхождений из строки поиска с выводом остатка.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД ПРОГРАММЫ

Название файла: main.py

```
import sys
from collections import deque

def solve_task1():
    T = input()
    n = int(input())
    patterns = []
    for i in range(n):
        patterns.append(input())

    print("=" * 60)
    print("STEP 1: BUILDING TRIE")
    print("=" * 60)

    son = [{}]
    go = [{}]
    fail = [0]
    term = [-1]
    pats = [[]]

    for pi, p in enumerate(patterns):
        cur = 0
        for c in p:
            if c not in son[cur]:
                son[cur][c] = len(son)
                son.append({})
                go.append({})
                fail.append(0)
                term.append(-1)
                pats.append([])
```

```

        print(f"      New node {len(son)-1}: parent={cur},
edge='{c}''")
        cur = son[cur][c]
        pats[cur].append(pi + 1)
        print(f"  Pattern {pi+1} '{p}' terminates at node {cur}")

    total_nodes = len(son)
    print(f"\nTrie built: {total_nodes} nodes")

    max_arcs = max(len(son[i]) for i in range(total_nodes))
    max_arcs_nodes = [i for i in range(total_nodes) if len(son[i])
== max_arcs]
    print(f"\n[VARIANT 5] Max arcs from one trie node: {max_arcs}")
    print(f"      Node(s) with max arcs: {max_arcs_nodes}")

    print("\n" + "=" * 60)
    print("STEP 2: BUILDING AUTOMATON (fail links + terminal links)
via BFS")
    print("=" * 60)

    q = deque()

    for c, s in son[0].items():
        fail[s] = 0
        go[0][c] = s
        q.append(s)

    all_chars = set()
    for node in son:
        all_chars |= set(node.keys())

    for c in all_chars:
        if c not in go[0]:
            go[0][c] = 0

    while q:
        v = q.popleft()

        f = fail[v]
        if pats[f]:
            term[v] = f
        elif term[f] != -1:
            term[v] = term[f]
        else:

```

```

        term[v] = -1

    print(f"  Node {v}: fail={fail[v]}, term={term[v]}, "
          f"trie_arcs={list(son[v].keys())},
patterns={pats[v]}")

    for c, s in son[v].items():
        fail[s] = go[fail[v]].get(c, 0)
        go[v][c] = s
        q.append(s)

    for c in all_chars:
        if c not in go[v]:
            go[v][c] = go[fail[v]].get(c, 0)

print("\n" + "=" * 60)
print("STEP 3: FULL AUTOMATON DESCRIPTION (every node)")
print("=" * 60)
for i in range(total_nodes):
    print(f"  Node {i:3d} | trie_children={son[i]} | "
          f"go={go[i]} | fail={fail[i]} | "
          f"term={term[i]} | patterns={pats[i]}")

print("\n" + "=" * 60)
print("STEP 4: SEARCHING TEXT")
print("=" * 60)
print(f"  Text = '{T}'")
print()

results = []
covered = [False] * len(T)
cur = 0

for i, c in enumerate(T):
    prev = cur
    cur = go[cur].get(c, 0)
    print(f"  i={i+1} char='{c}': state {prev} -> {cur}")

    tmp = cur
    while tmp != 0:
        if pats[tmp]:
            for pi in pats[tmp]:
                plen = len(patterns[pi - 1])
                start = i - plen + 2

```

```

        results.append((start, pi))
        print(f"        [via node {tmp}] Found pattern {pi}")
    """
        f"'{patterns[pi-1]}' at position {start}")
    for x in range(i - plen + 1, i + 1):
        covered[x] = True
    if tmp == cur:
        tmp = term[tmp]
    else:
        tmp = term[tmp]
    if tmp == -1:
        break

    print("\n" + "=" * 60)
    print("FINAL RESULTS (sorted by position then pattern number)")
    print("=" * 60)
    results.sort()
    for pos, pi in results:
        print(f"{pos} {pi}")

    remainder = ''.join(T[i] for i in range(len(T)) if not
covered[i])
    print(f"\n[VARIANT 5] Remainder after cutting all found patterns:
'{remainder}'")

def solve_task2():
    T = input()
    P = input()
    wildcard = input()

    print("\n" + "=" * 60)
    print("WILDCARD SEARCH")
    print("=" * 60)
    print(f"    Text:      '{T}'")
    print(f"    Pattern:   '{P}'")
    print(f"    Wildcard:  '{wildcard}'")

    print("\n--- Step 1: Splitting pattern into chunks ---")
    chunks = []
    i = 0
    while i < len(P):
        if P[i] == wildcard:
            i += 1
        else:

```



```

        j = i
        while j < len(P) and P[j] != wildcard:
            j += 1
        chunks.append((i, P[i:j]))
        print(f"    Chunk: '{P[i:j]}' at pattern position {i}")
        i = j

k = len(chunks)
if k == 0:
    pat_len = len(P)
    for pos in range(1, len(T) - pat_len + 2):
        print(pos)
    return

print("\n--- Step 2: Building Aho-Corasick on chunks ---")

son = [{}]
go = [{}]
fail = [0]
term = [-1]
pats = [[]]

for ci, (lpos, chunk) in enumerate(chunks):
    cur = 0
    for c in chunk:
        if c not in son[cur]:
            son[cur][c] = len(son)
            son.append({})
            go.append({})
            fail.append(0)
            term.append(-1)
            pats.append([])
            print(f"        New node {len(son)-1}: parent={cur},
edge='{c}'")
            cur = son[cur][c]
        pats[cur].append((ci, lpos, len(chunk)))
    print(f"    Chunk '{chunk}' terminates at node {cur}")

all_chars = set()
for node in son:
    all_chars |= set(node.keys())

q = deque()
for c, s in son[0].items():

```

```

        fail[s] = 0
        go[0][c] = s
        q.append(s)
    for c in all_chars:
        if c not in go[0]:
            go[0][c] = 0

    while q:
        v = q.popleft()
        f = fail[v]
        if pats[f]:
            term[v] = f
        elif term[f] != -1:
            term[v] = term[f]
        else:
            term[v] = -1

        for c, s in son[v].items():
            fail[s] = go[fail[v]].get(c, 0)
            go[v][c] = s
            q.append(s)
        for c in all_chars:
            if c not in go[v]:
                go[v][c] = go[fail[v]].get(c, 0)

    print("\n--- Step 3: Searching text for chunks ---")
    n = len(T)
    pat_len = len(P)
    C = [0] * (n + 2)

    cur = 0
    for i, c in enumerate(T):
        prev = cur
        cur = go[cur].get(c, 0)
        print(f"  i={i+1} char='{c}': state {prev} -> {cur}")

        tmp = cur
        while tmp != 0:
            if pats[tmp]:
                for (ci, lpos, clen) in pats[tmp]:
                    start = i - clen - lpos + 1
                    if 0 <= start and start + pat_len <= n:
                        C[start] += 1
                        print(f"      Chunk '{chunks[ci][1]}' found, ")

```

```

        f"pattern would start at {start+1} "
        f"(C[{start}]=={C[start]})")
    if tmp == cur:
        tmp = term[tmp]
    else:
        tmp = term[tmp]
    if tmp == -1:
        break

print("\n--- Step 4: Checking counter array ---")
results = []
for i in range(n):
    if C[i] == k and i + pat_len <= n:
        results.append(i + 1)
        print(f" Position {i+1}: C[{i}]=={C[i]} == {k} -> MATCH")

print("\n" + "=" * 60)
print("FINAL RESULTS")
print("=" * 60)
for pos in results:
    print(pos)

print("Выберите задание (1 или 2):")
choice = input().strip()

if choice == "1":
    solve_task1()
elif choice == "2":
    solve_task2()
else:
    print("Неверный выбор. Введите 1 или 2.")

```